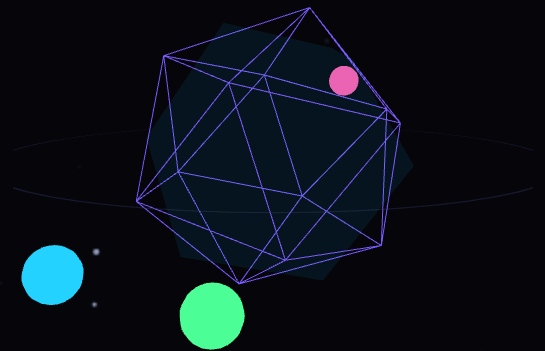




MCP 야호~

MCP(Model Context Protocol)를 처음부터 직접 만들어보며 배우는 학습자료입니다.
"MCP가 대체 뭐고, 왜 필요하고, 어떻게 만드는가"를 개념 → 실전 순서로 전달하는
것이 목표입니다.

 제작자 · 박건호

 gunh0

 LinkedIn

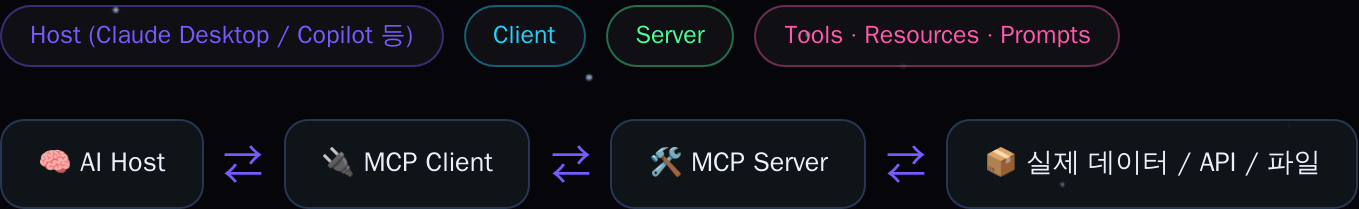
▼ SCROLL TO START QUEST ▼



▶ QUEST 01

MCP(Model Context Protocol)가 뭔가요?

MCP는 Anthropic이 만든 오픈 표준 프로토콜로, AI 모델이 외부 데이터·도구·서비스에 "표준화된 방식"으로 연결하도록 해줍니다. 흔히 "AI를 위한 USB-C"라고 비유합니다 — 각 데이터소스마다 전용 케이블(커스텀 연동 코드)을 만들 필요 없이, 하나의 규격만 지키면 됩니다.



왜 필요한가요?

AI마다 커스텀 연동을 만들면 **N개 AI × M개 도구** 만큼 코드가 늘어납니다(N×M 문제). MCP는 서버 하나만 잘 만들면 어떤 MCP 호환 AI에서도 재사용되므로 **N+M** 로 줄어듭니다.

3가지 핵심 개념

Tool: AI가 호출해 실행하는 함수(예: 파일 검색, API 호출)
Resource: AI가 읽어올 수 있는 데이터(문서, DB row 등)
Prompt: 자주 쓰는 프롬프트 템플릿

대표 예시 서버

파일시스템 MCP 서버, GitHub MCP 서버, Slack MCP 서버, DB(Postgres) MCP 서버 등 이미 공식/커뮤니티 구현체가 다양하게 존재합니다.



▶ QUEST 02

MCP는 프로토콜 — 그럼 "MCP 서버"는 뭔가요?

HTTP와 웹 서버의 관계 그대로입니다. HTTP가 "요청·응답을 이런 형식으로 주고받자"는 약속이고 웹 서버가 그 약속대로 말하는 프로그램이듯, MCP 서버는 자신의 Tool·Resource·Prompt를 MCP 규격으로 노출할 줄 아는 프로그램입니다. 뒤에서 만들 server.py 하나가 곧 MCP 서버예요.



📦 MCP 메시지 규격 — JSON-RPC

규격은 하나 그대로, **전송 방식만 두 가지 중에서 선택**합니다. HTTP 서버와 달리 꼭 포트를 열 필요가 없어요.

① stdio 전송 — 포트 없음

AI 호스트가 MCP 서버를 자식 프로세스로 직접 실행하고 stdin/stdout 파이프로 대화합니다. 100% 로컬에서 동작해요.

예) Claude Desktop 설정에 `server.py` 실행 명령을 등록해두면, 대화가 시작될 때 Claude가 서버를 자식 프로세스로 직접 띄워 내 PC의 파일을 검색합니다.

🚫 포트 안 열림

🔒 망분리 환경 OK

② Streamable HTTP 전송 — 포트 대기

원격 서버로 띄워두고 여러 클라이언트가 접속합니다. 이때는 MCP가 HTTP 위에 얹혀서 동작해요.

예) 내부망 공용 서버에 **Confluence MCP** 서버를 하나 띄워두면, 팀원 모두가 각자의 AI 호스트에 URL만 등록해서 같은 서버에 접속합니다.

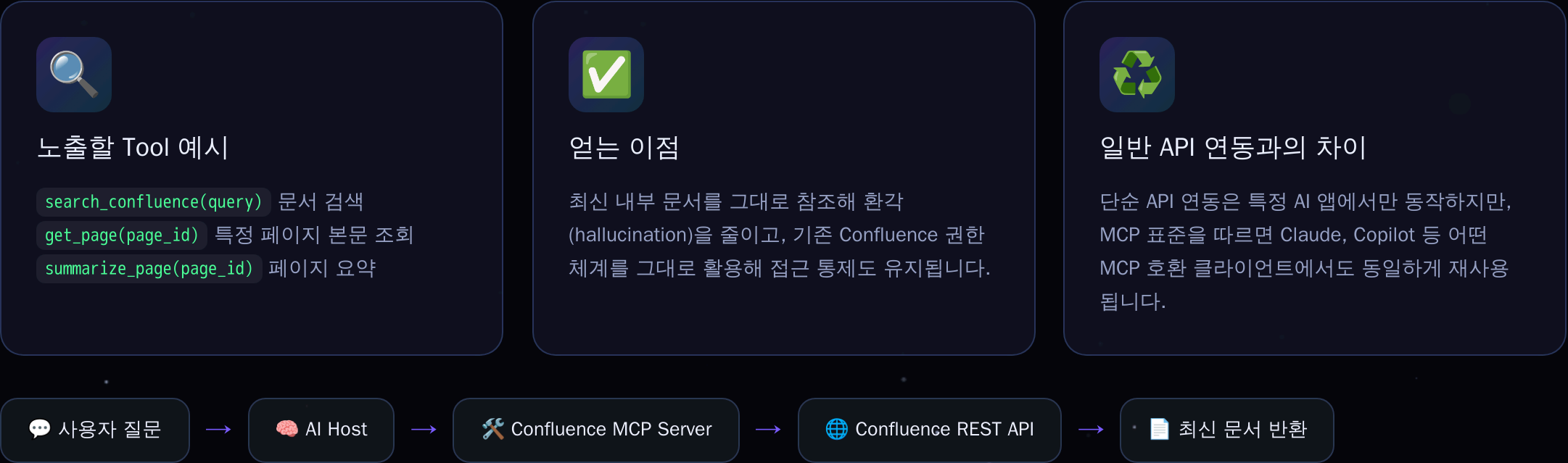
📡 네트워크 포트 오픈

👥 여러 클라이언트 공유

▶ QUEST 03

Confluence를 MCP 서버처럼 활용하기

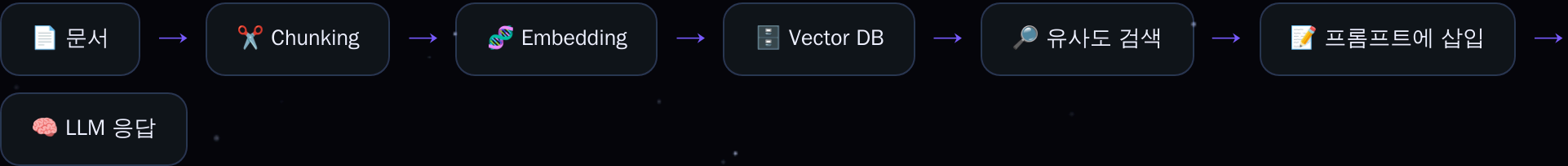
내부망 위키(Confluence)의 REST API를 감싸서 MCP Server로 노출하면, AI가 최신 내부 문서를 실시간으로 검색·조회하며 답변에 활용할 수 있습니다. "AI에게 조직의 지식을 읽는 눈을 달아주는 것"이라 생각하면 쉽습니다.



▶ QUEST 04

RAG 구성하기 — MCP와 뭐가 다른가요?

RAG(Retrieval-Augmented Generation)는 "질문과 관련된 문서를 미리 찾아서 프롬프트에 끼워 넣는 기법"입니다. MCP는 "AI가 외부 도구·데이터에 접근하는 방식을 표준화한 프로토콜"이라는 점에서 다루는 차원이 다릅니다.



구분	RAG	MCP
정체	데이터를 검색해 프롬프트에 주입하는 기법 / 파이프라인	도구·데이터 접근을 위한 표준 통신 프로토콜
주요 관심사	임베딩, 벡터DB, 유사도 검색 품질	Tool/Resource/Prompt 노출, 클라이언트-서버 통신 규격
할 수 있는 것	주로 "읽기" 중심 (관련 문서 검색)	읽기 + 실행(Tool 호출) 모두 가능 (예: 점검 스크립트 실행)
재사용성	보통 특정 앱/파이프라인에 종속	MCP 호환 클라이언트라면 어디서든 재사용
관계	서로 대체 관계가 아니라 보완 관계입니다. RAG 파이프라인 자체를 MCP Tool(search())로 감싸서 노출하면, RAG를 여러 AI 클라이언트에서 표준적으로 쓸 수 있습니다.	

▶ QUEST 05

.claude로 프로젝트 컨텍스트 관리하기

Claude Code의 .claude는 두 가지가 있습니다 — 개인 전역 설정(~/.claude)과 프로젝트 레벨(<레포>/.claude + CLAUDE.md). 여기서 다루는 것은 후자입니다. 프로젝트의 규칙·컨벤션·자주 쓰는 명령, 커스텀 skills·agents를 레포에 커밋해두면 클론한 팀원 누구나 같은 컨텍스트로 에이전트를 쓰게 됩니다.



▶ QUEST 06

직접 구현해보기: Python + Docker 로컬 MCP

공식 MCP Python SDK로 서버를 직접 구현하고, Docker로 패키징해서 누구나 동일한 방식으로 로컬에서 실행·검증할 수 있게 표준화합니다. 인터넷과 분리된 망분리(프라이빗 네트워크) 환경에서도 그대로 동작하는 구성이 목표입니다.



MCP Python SDK

공식 SDK의 `@server.tool()` 데코레이터로 보안 점검·자료 정리·보고서 초안 작성 같은 보안 업무용 Tool을 함수 하나로 정의합니다.



Docker 표준화

`Dockerfile` 하나로 실행 환경을 고정해, "내 PC에선 되는데?" 문제 없이 팀원 모두 동일한 결과를 재현합니다.



업무 Tool 조합

클라우드 보안 점검 자동화, 점검 결과 정리, 보고서 초안 생성 Tool을 조합해 반복 업무를 줄이는 것이 1차 목표입니다.

▶ 간단한 서버 골격 예시 — server.py + Dockerfile

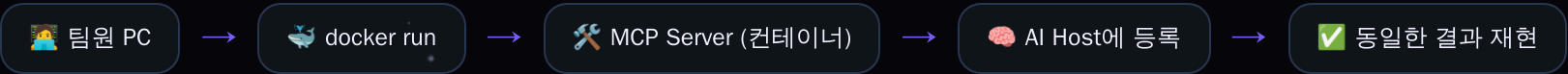
```
# server.py - 공식 MCP Python SDK 예시
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("cloud-security-mcp")

@mcp.tool()
def check_security_group(sg_id: str) -> dict:
    """지정한 보안그룹의 위험한 인바운드 룰을 점검한다"""
    # TODO: 클라우드 API 호출 후 점검 결과 반환
    return {"sg_id": sg_id, "risky_rules": []}

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

```
# Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "server.py"]
```



▶ FINAL QUEST

학습 로드맵

용어 정리에서 시작해 나만의 MCP 서버를 만들고, Anthropic이 권고하는 에이전트 엔지니어링으로 나아가는 여정입니다.

- STEP 1

개념·용어 정리

Host·Client·Server 구조, Tool/Resource/Prompt, JSON-RPC 규격과 두 가지 전송 방식(stdio vs Streamable HTTP), RAG, `.claude` 와의 역할 구분까지 언어부터 잡습니다.
- STEP 2

간단한 MCP 서버 만들기

Python SDK로 Tool 1~2개짜리 서버를 만들고 Claude Desktop/Claude Code에 stdio로 연결, MCP Inspector로 호출 흐름을 눈으로 확인합니다.
- STEP 3

Docker 패키징 & 망분리 환경 이식

실행 환경을 Dockerfile로 고정하고, 내부망 데이터소스(Confluence 등)와 연동해 보안 점검·정리·보고 Tool 조합을 실제 업무 흐름에 엮어봅니다.
- STEP 4

Tool을 "에이전트답게" 설계하기

Anthropic의 Tool 설계 권고 적용 — 명확한 설명과 좁은 책임의 Tool, 토큰 효율적인 응답, 모델이 스스로 복구할 수 있는 에러 메시지.

▶ 그 다음은? — 심층 학습 트랙



컨텍스트 엔지니어링

한정된 컨텍스트 창을 예산처럼 관리하기 — 필요한 순간에만 불러오는 just-in-time 컨텍스트, 긴 대화의 압축(compaction), 검색과 주입의 균형.



하네스 엔지니어링

모델을 감싸는 에이전트 루프 설계 — 도구 호출 반복, 권한·승인 게이트, 서브에이전트 분업, `.claude` (CLAUDE.md-skills)와 MCP의 조합.



평가와 보안

Tool 품질을 평가(evals)로 측정하며 반복 개선하고, 최소 권한 원칙과 prompt injection 같은 MCP 특유의 위협 대응까지 다룹니다.